

高层追击策略：算法思路与奖励函数

1 算法选择

1.1 整体思路

采用两阶段训练：

1. 行为克隆 (Behavior Cloning, BC)：用传统追击策略（Lead Pursuit、Expert 等）的成功 rollout 数据，通过监督学习预训练高层策略网络，提供合理的初始化；
2. 强化学习微调 (RL Fine-tuning)：以 BC 权重为起点，用 PPO 在线交互训练，突破传统策略的性能上限。

1.2 为什么先 BC 再 RL

我思考直接 RL 失败的原因：连续动作空间 $[-1, 1]^2$ 中随机探索几乎无法产生有效追击行为；BC 预训练解决了冷启动问题——给 RL 一个“已经会追”的起点。

1.3 Phase 1: OfflineRL

数据来源 混合多种传统策略的成功 episode (captured=True)。

先不接入底层策略，让高层专注学会选 subgoal 追击，在高层设计好的奖励函数下计算每一时刻状态、动作对应的奖励-后续 RL 阶段再微调接入底层执行的状态分布。

目前高层用传统策略已经能达到 80% 以上的 catch_rate，也许作为专家已经足够了？

1.4 Phase 2: PPO + Online 微调

算法 Proximal Policy Optimization (PPO)，加入对 BC 策略的 KL 散度惩罚：

$$\mathcal{L} = \mathcal{L}_{\text{PPO}}(\theta) + \beta \cdot D_{\text{KL}}(\pi_{\theta} \parallel \pi_{\text{BC}}), \quad (1)$$

β 从 β_0 线性退火至 0，让策略逐渐脱离 BC 约束。

课程学习 逃脱者难度递进：simple $\rightarrow$ medium $\rightarrow$ hard。

2 奖励函数设计

2.1 设计原则

高层策略的任务是：给定当前状态，输出一个 subgoal 供低层跟踪。奖励函数需要回答两个问题：

1. 追得好不好？——subgoal 是否让追击者有效靠近逃脱者；
2. 走得安不安全？——前往 subgoal 的路径是否远离障碍物，否则感觉画出来很蠢。

注意：传统策略（Lead、PN、Expert 等）本身不考虑障碍物，只做纯几何制导；低层控制器也不做避障，只跟踪 subgoal。虽然环境有后处理（_push_subgoal_out_of_obstacles 方向扫描修正），但这只保证 subgoal 不在障碍物内，不保证前往 subgoal 的路径安全。因此避障必须作为奖励信号，让高层策略主动学会绕行。

2.2 总体结构

目前我想高层的奖励由四项组成，每项职责单一、互不冲突：

$$r_t = r_t^{\text{event}} + r_t^{\text{shaping}} + r_t^{\text{obs}} + r_t^{\text{time}}. \quad (2)$$

2.3 第一项：完成任务的奖励 r^{event} （稀疏）

$$r_t^{\text{event}} = \begin{cases} +R_{\text{catch}} & \text{捕获 } (d_{pe} \leq d_{\text{catch}}), \\ -R_{\text{fail}} & \text{碰撞障碍物或出界}, \\ 0 & \text{其他}. \end{cases} \quad (3)$$

仅在 episode 结束时触发。定义任务的最终目标。

2.4 第二项：距离整形 r^{shaping} （密集，管”追”）

采用 PBRS（Potential-Based Reward Shaping, Ng et al. 1999），理论保证不改变最优策略。

势函数

$$\Phi(s) = -\|p_p - p_e\|_2 \quad (\text{负的追逃距离}) \quad (4)$$

整形奖励

$$r_t^{\text{shaping}} = \alpha(\gamma \Phi(s_{t+1}) - \Phi(s_t)). \quad (5)$$

为什么这一项隐式评价了 **subgoal 质量** 高层选 subgoal $\rightarrow$ 低层执行若干步 $\rightarrow$ 追击者到达新位置 $\rightarrow$ 距离变化 $\rightarrow$ PBRs 奖励。好的 subgoal 让距离缩短（正奖励），差的 subgoal 让距离不变或增大（零或负奖励）。不需要显式比较”RL 的 subgoal 与专家 subgoal 的差距”——那样会限制策略只能模仿专家，无法发现更优路径？

2.5 第三项：避障惩罚 r^{obs} （密集，管”绕”）

$$r_t^{\text{obs}} = -c \cdot \max\left(0, 1 - \frac{d_{\text{nearest}}}{d_{\text{safe}}}\right). \quad (6)$$

其中 d_{nearest} 为追击者到最近障碍物边缘的距离。

- $d_{\text{nearest}} \geq d_{\text{safe}}$: 惩罚为 0，不影响正常追击；
- $d_{\text{nearest}} \rightarrow 0$: 惩罚 $-c$ 。

为什么需要这一项

- 碰撞的事件惩罚 $-R_{\text{fail}}$ 太稀疏，策略难以学到”提前绕行”；
- subgoal 后处理只修正了目标点，但到达目标点的路径仍可能贴着障碍物；
- 这一项提供连续的梯度信号：离障碍物越近，惩罚越大，鼓励高层主动选择远离障碍物的 subgoal。

2.6 第四项：时间惩罚 r^{time} （密集，管”快”）

$$r_t^{\text{time}} = -\lambda. \quad (7)$$

每步固定扣 λ 。需要保证整个 episode 的累计时间惩罚远小于捕获奖励 R_{catch} 。

2.7 各项量级与职责总结

奖励项	类型	典型量级	职责
事件奖励	稀疏	$+R_{\text{catch}} / -R_{\text{fail}}$	任务终局目标
距离整形	密集	取决于 α, γ	引导追击（评价 subgoal 质量）
避障惩罚	密集	$0 \sim -c/\text{步}$	引导绕行（路径安全）
时间惩罚	密集	$-\lambda/\text{步}$	防止拖延

四项奖励各管一件事：追、绕、终局、快。信号不冲突，需要调节的超参包括： $R_{\text{catch}}, R_{\text{fail}}, d_{\text{catch}}, \alpha, \gamma, c, d_{\text{safe}}, \lambda, \beta_0$ 。